DuocUC | ONLINE



Guía de aprendizaje

Introducción a Spring Batch y sus componentes

Exp 1 – Semana 1

Desarrollo Backend III (PBY2203)

Facilitador disciplinar: Ana Karina Villagrán Ibarra

Asesor par: Ignacio Villaroel

Índice

Introducción a la semana....3
Resultado de aprendizaje....4
Conceptos relevantes....4
Preguntas activadoras....5
Actividad....5
Conociendo Spring Batch....6
Qué es Spring Batch....7
Características principales....8
Evolución histórica de Spring Batch....8
Arquitectura de Spring Batch....9
Configurando un proyecto en Spring Batch....12
Ejecutando un “Hello World”....14
Leyendo, transformando y persistiendo datos....17
Estrategias de migración de sistemas legacy....21
Cierre de la semana....24
Referencias....25
Lecturas de la semana....25
Apuntes....26

Introducción a la semana



En esta semana identificarás los conceptos fundamentales de Spring Batch y sus componentes claves, enfocados en la migración de sistemas legacy. Explorarás cómo esta tecnología te permite desarrollar soluciones batch eficientes para la transformación de datos de sistemas antiguos, como aquellos basados en COBOL o shell scripts.

En esta guía de aprendizaje encontrarás:

- Cómo diseñar una arquitectura basada en Spring Batch para la migración de sistemas legacy.
- Cómo ejecutar jobs y steps en Spring Batch para transformar datos de entrada y generar una salida procesada.
- Qué beneficios ofrece Spring Batch en comparación con otras herramientas de migración de datos.

Hasta ahora, eres capaz de ofrecer una variedad de soluciones a requerimientos y casos específicos en tus proyectos. En adición, gracias a los contenidos de esta experiencia de aprendizaje, también podrás optimizar procesos de migración de datos desde sistemas antiguos, guiándote por los principios y buenas prácticas de Spring Batch. Además, aprenderás a administrar mejor los recursos del sistema, utilizando herramientas modernas que están en alta demanda para proyectos de migración y transformación de datos.

Resultado de aprendizaje

El estudiante será capaz de:

RA1. Desarrolla componentes batch para la migración de sistemas antiguos basados en cobol/shell.

Indicador de logro:

IL1. Diseña una arquitectura de Spring Batch, definiendo su rol en la migración de sistemas legacy.

IL2. Ejecuta jobs y steps en Spring Batch, para transformar datos de entrada y generar una salida.

Conceptos relevantes

Spring Batch	Jobs	Steps
Legacy	Cobol / Shell	ItemReader
ItemProcessor	ItemWriter	

Preguntas activadoras

- ¿Has oído hablar de procesos batch o por lotes?
- ¿Qué crees que significa procesar datos en "lotes" o "batch"?
- ¿Por qué crees que sería importante dividir un gran proceso en pasos más pequeños?
- ¿Te ha tocado trabajar con grandes volúmenes de datos? ¿Cómo imaginas que se podrían organizar mejor para ser procesados de manera eficiente?

Actividad

En esta primera semana, realizarás una actividad formativa grupal, llamada "Analizando la arquitectura batch para procesar datos", donde, a través de un caso ficticio, diseñarás una arquitectura basada en Spring Batch, definiendo su rol en la migración de sistemas legacy desarrollados en Cobol o Shell scripts. Aprenderás a configurar y ejecutar jobs y steps para transformar datos de entrada y generar una salida procesada. Además, aplicarás estrategias para manejar errores y excepciones, asegurando la correcta ejecución de los sistemas batch.

Conociendo Spring Batch

Imagina que en tu empresa hay un sistema muy antiguo desarrollado en COBOL que aún se usa para realizar tareas críticas. Sin embargo, actualizarlo o integrarlo con nuevas tecnologías es complicado debido a su antigüedad y las herramientas obsoletas con las que fue creado. Este tipo de sistema se conoce como **"sistema legacy".**

Aunque es fundamental para el negocio, su mantenimiento y mejora presentan desafíos. Aquí es donde entra en juego Spring Batch: esta herramienta te permite modernizar los procesos batch de ese software en COBOL o Shell scripts, ayudándote a automatizar y optimizar el procesamiento de datos sin tener que reemplazar el sistema completo. Así, puedes hacer que el trabajo sea más eficiente y compatible con las tecnologías actuales, preservando la funcionalidad clave del sistema legacy.

Links de Interés

Un sistema legacy, o sistema heredado, es una tecnología, aplicación o software antiguo que sigue en uso dentro de una organización porque continúa cumpliendo con sus funciones originales. Para obtener más información, consulta el siguiente enlace: https://www.stackscale.com/es/blog/sistemas-legacy/

Qué es Spring Batch



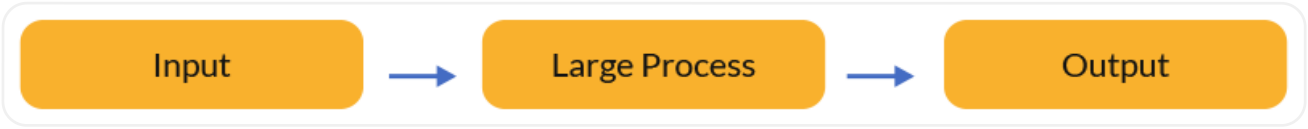
Spring Batch es un framework que facilita el procesamiento de grandes volúmenes de datos a través de trabajos por lotes (batch processing). Está diseñado para automatizar la ejecución de tareas repetitivas y estructuradas en un sistema, siendo especialmente útil para la migración de sistemas legacy y la transformación de datos.

El framework se estructura en jobs (trabajos) que contienen steps (pasos), cada uno enfocado en una tarea específica, como la lectura, transformación y escritura de datos. Los datos pueden provenir de diversas fuentes, como bases de datos, archivos, o sistemas legacy, y ser procesados de manera eficiente.

Imagina, por ejemplo, los datos de los habitantes de un país, que requieren conocer el pago de sus impuestos a partir de una serie de consultas. Esto sería un proceso Batch y una estructura básica de este proceso, sería de la siguiente manera:

Figura 1

Componentes básicos de un proceso



Nota. Este diagrama muestra un flujo de procesamiento donde tienes una entrada de datos ("Input") que pasa por un proceso extenso o complejo ("Large Process") antes de generar un resultado final ("Output").

Características principales

Spring Batch ofrece una serie de características que permiten la gestión eficiente de procesos por lotes. Algunas de las más destacadas son:

- Modularidad: cada proceso batch se divide en pasos independientes (steps), lo que permite una organización modular y clara del flujo de trabajo.
- Escalabilidad: permite manejar grandes volúmenes de datos y optimiza los recursos, siendo ideal para migraciones de sistemas legacy.
- Reinicio de Jobs: si un job falla, Spring Batch puede reanudar su ejecución desde el último paso completado, lo que asegura que no se pierda el progreso en caso de errores.
- Transacciones: los procesos batch están controlados por transacciones, lo que asegura la integridad de los datos durante la lectura, procesamiento y escritura.
- Manejo de errores: ofrece un manejo robusto de excepciones y la capacidad de gestionar errores sin detener todo el proceso.
- Integración con Spring: se integra fácilmente con otras tecnologías del ecosistema Spring, como Spring Boot y Spring Cloud.

Evolución histórica de Spring Batch

A pesar del enfoque creciente en arquitecturas basadas en la web y microservicios, ha habido una falta notable de marcos reutilizables para el procesamiento por lotes en Java, lo que ha llevado a muchas soluciones internas y únicas dentro de las empresas.

Para abordar esta necesidad, SpringSource (ahora VMware) y Accenture colaboraron, combinaron la experiencia de Accenture en la implementación de arquitecturas por lotes, con la capacidad técnica de SpringSource, para desarrollar Spring Batch, un marco que llena este vacío en el procesamiento por lotes en Java empresarial.

Accenture aportó con su vasta experiencia en arquitecturas por lotes, que abarcaban desde COBOL en mainframes hasta Java en plataformas modernas al proyecto Spring Batch. Esta colaboración buscó estandarizar herramientas y enfoques para el procesamiento por lotes, ofreciendo una solución robusta y probada para entornos empresariales.

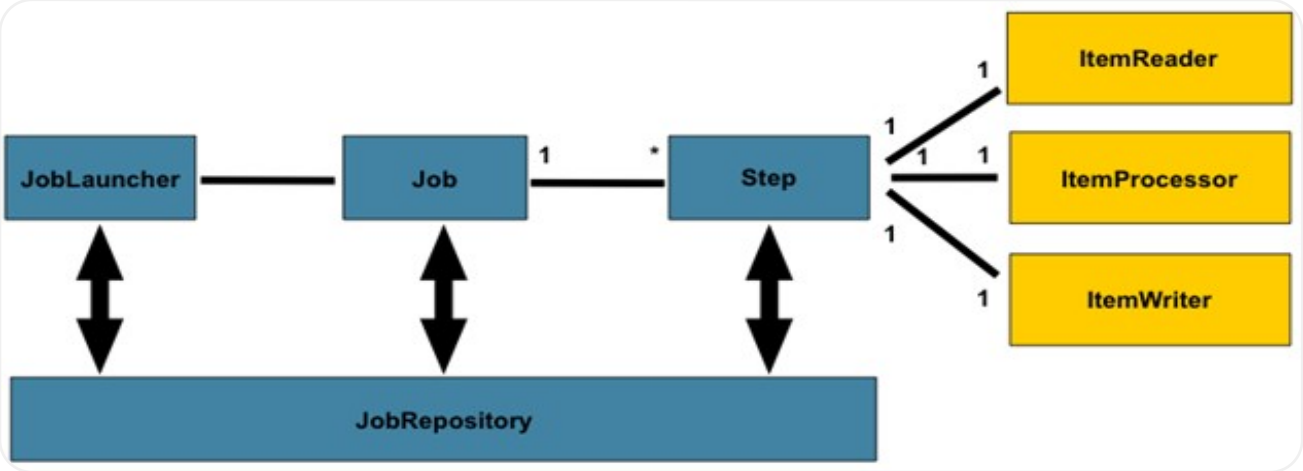
Arquitectura de Spring Batch

Para comenzar a utilizar Spring Batch, se deben identificar los componentes clave del procesamiento por lotes.

La siguiente figura se utilizó como una arquitectura de referencia para cualquier framework que deseara implementar un procesamiento por lotes:

Figura 2

Estereotipo Batch



Nota. Diagrama que muestra la estructura de un trabajo batch en Spring Batch, donde el JobLauncher ejecuta un Job compuesto por pasos (Step). *SpringSource (s.f.). Chapter 3. The Domain Language of Batch.* https://docs.spring.io/spring-batch/docs/2.2.x/reference/html/domain.html

Básicamente, Spring Batch utiliza esta arquitectura para crear sus propias clases, clases abstractas e interfaces en Java.

A continuación, revisarás los conceptos que crea Spring Batch para implementar la arquitectura de referencia “Estereotipo Batch”.

Componentes clave para una migración de sistemas legacy

- **Job:** representa el proceso completo de migración. Un job está compuesto por varios steps, cada uno de los cuales realiza una tarea específica (lectura, transformación o escritura de datos).
- **JobLauncher:** es la entidad que crea al contexto para ejecutar un Job
- **Step**: un step es una fase independiente del proceso batch. Por ejemplo, un step puede encargarse de leer datos de un sistema legacy, otro de transformarlos, y otro de escribirlos en el nuevo sistema.
- **ItemReader:** es el componente encargado de leer los datos de entrada. En una migración, puede leer desde una base de datos antigua, archivos CSV, XML o JSON, o incluso de servicios web legacy.
- **ItemProcessor:** procesa los datos leídos, aplicando transformaciones necesarias. Este es el lugar ideal para convertir los formatos legacy a formatos más modernos o para realizar la normalización de datos.
- **ItemWriter:** escribe los datos procesados en el nuevo sistema. Puede escribir en una base de datos moderna, un sistema NoSQL, o incluso invocar APIs para almacenar los datos en un servicio externo.

Links de Interés

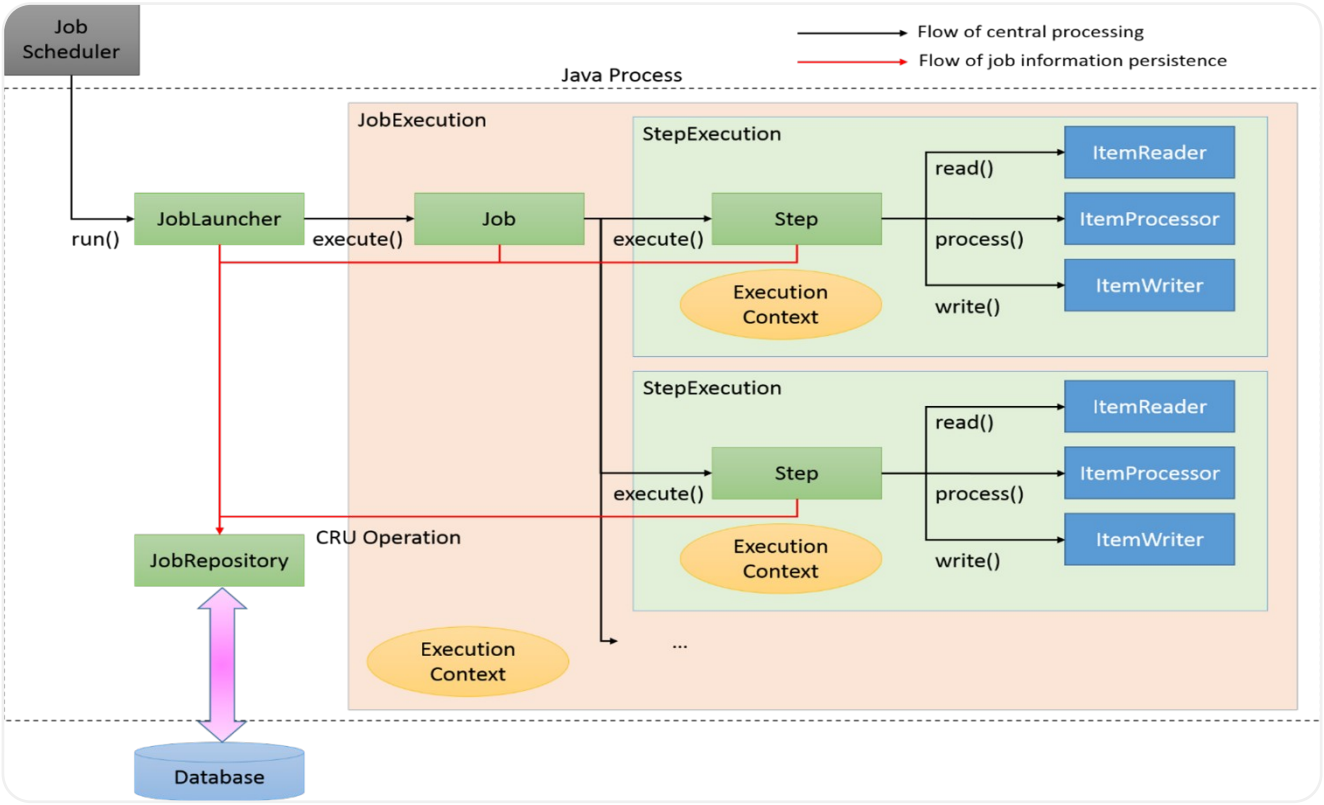
Existen otros conceptos que incorporarás a medida que la complejidad de los procesos que requieras migrar aumente. Para tener un mapa completo de las clases disponibles para armar procesos batch, considera visitar el siguiente enlace: https://docs.spring.io/spring-batch/docs/current/api/allclasses-index.html

Lo importante es que seas capaz de identificar los conceptos anteriores en el diagrama general de un proceso batch.

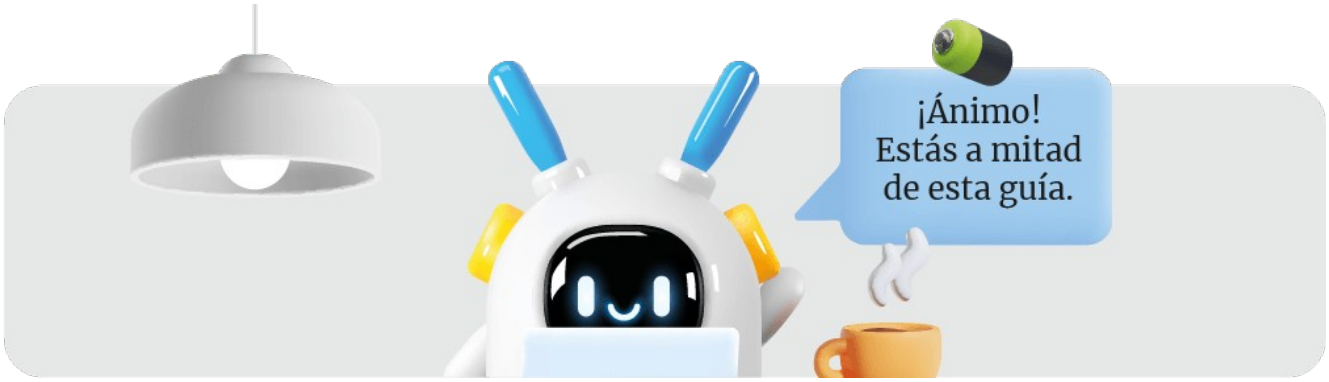
La siguiente figura será el “mapa” para ubicarte en el contexto general de Spring Batch:

Figura 3

Flujo general de un proceso batch



Nota. La imagen muestra el flujo general de un proceso batch, brindando una explicación sobre cómo administrar los metadatos del estado de ejecución de los trabajos, los componentes principales de Spring Batch y el flujo general del proceso. *Terasoluna (s.f.). Spring Batch Architecture.* https://terasoluna-batch.github.io/guideline/5.0.0.RELEASE/en/Ch02_SpringBatchArchitecture.html


¡Ánimo!
Estás a mitad
de esta guía.

Configurando un proyecto en Spring Batch

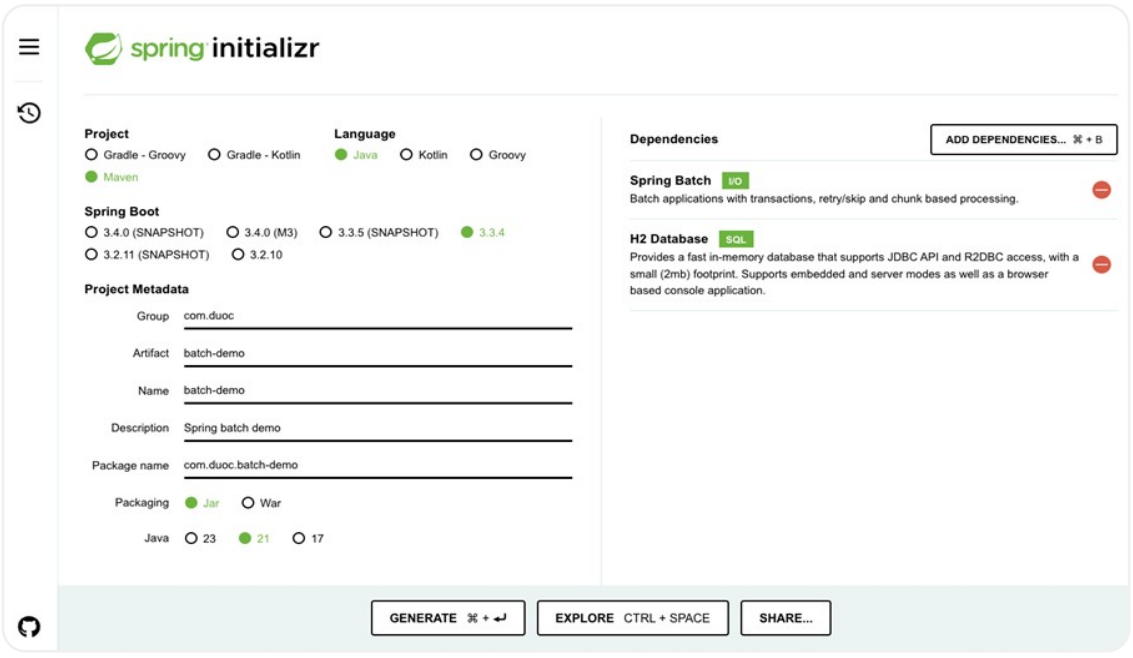
Lo primero que debes realizar para crear un proyecto en Spring batch es iniciar el navegador de internet e ingresar a: https://start.spring.io/

Importante

Para efectos de generar una actividad práctica, puedes hacer clic en el siguiente link con los parámetros y dependencias ya generados: Initializr base spring batch

Figura 4

Interfaz de Spring Initializr



Nota. Parámetros de creación de proyecto. SpringSource (2002). Spring Framework (6.1.12). [Software]. *Spring Initializr.* https://start.spring.io/

Una vez generado el proyecto base, sigue los siguientes pasos para poder ejecutar tu primer *Hello World* con Spring Batch.

- **Paso 1:** Crea en tu clase main un Job, un Step y un JobLauncher.
- **Paso 2:** Luego asegúrate de configurar tu application.properties de forma correcta.
- **Paso 3:** Elimina los unit-test. De momento para tu primer “Hello World” no serán necesarios.
- **Paso 4:** Lanza la rutina.

Link de interés

Este enlace te ayudará a realizar los pasos recién mencionados:
https://ava.duoc.cl/bbcswebdav/xid-5048769_1

Podrás acceder a los siguientes elementos para llevarlos a cabo:

- **Paso 1:** Referencia HelloWorldJobConfiguration.
- **Paso 2:** Referencia application.properties.
- **Paso 3:** Instrucciones para poder ejecutar tu aplicación.

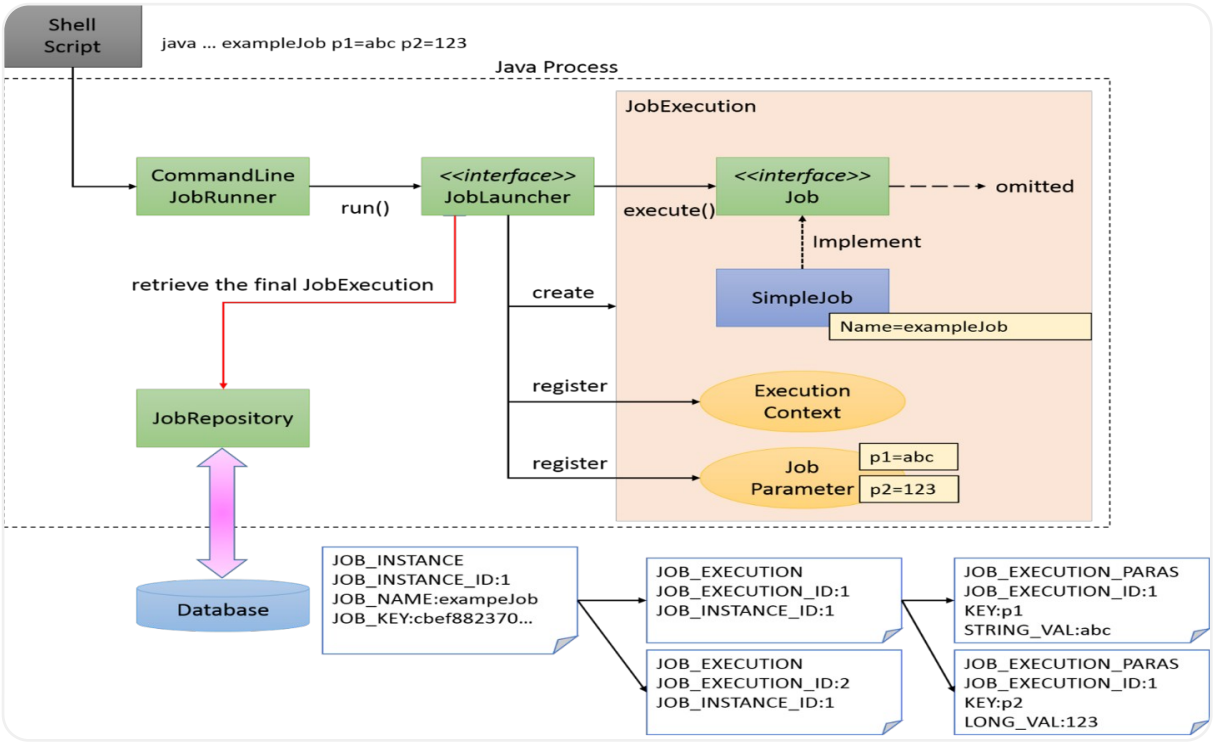
Ejecutando un “Hello World”

Una vez completado el paso 4, ya tendrás tu primer proceso Spring Batch, pero ¿cómo funciona?

Como puedes ver en la Figura 5, cuando se ejecuta un batch, existe un flujo de ejecución, que comienza con un script de Shell invocando el CommandLineJobRunner. Este, a su vez, utiliza el JobLauncher para ejecutar un Job definido, creando una instancia de JobExecution. El Job, implementado como un SimpleJob, usa un contexto de ejecución y parámetros específicos para la ejecución. La información del trabajo (instancia, ejecución y parámetros) se registra en el JobRepository, que persiste estos datos en una base de datos para el seguimiento del estado y la gestión del historial de ejecuciones.

Figura 5

Ejecución de un batch



Nota. El diagrama muestra cómo Spring Batch ejecuta un trabajo, gestionando instancias, parámetros y contexto, y persiste el estado en la base de datos. *Terasoluna (s.f.). Spring Batch Architecture.*

https://terasoluna-batch.github.io/guideline/5.0.0.RELEASE/en/Ch02_SpringBatchArchitecture.html

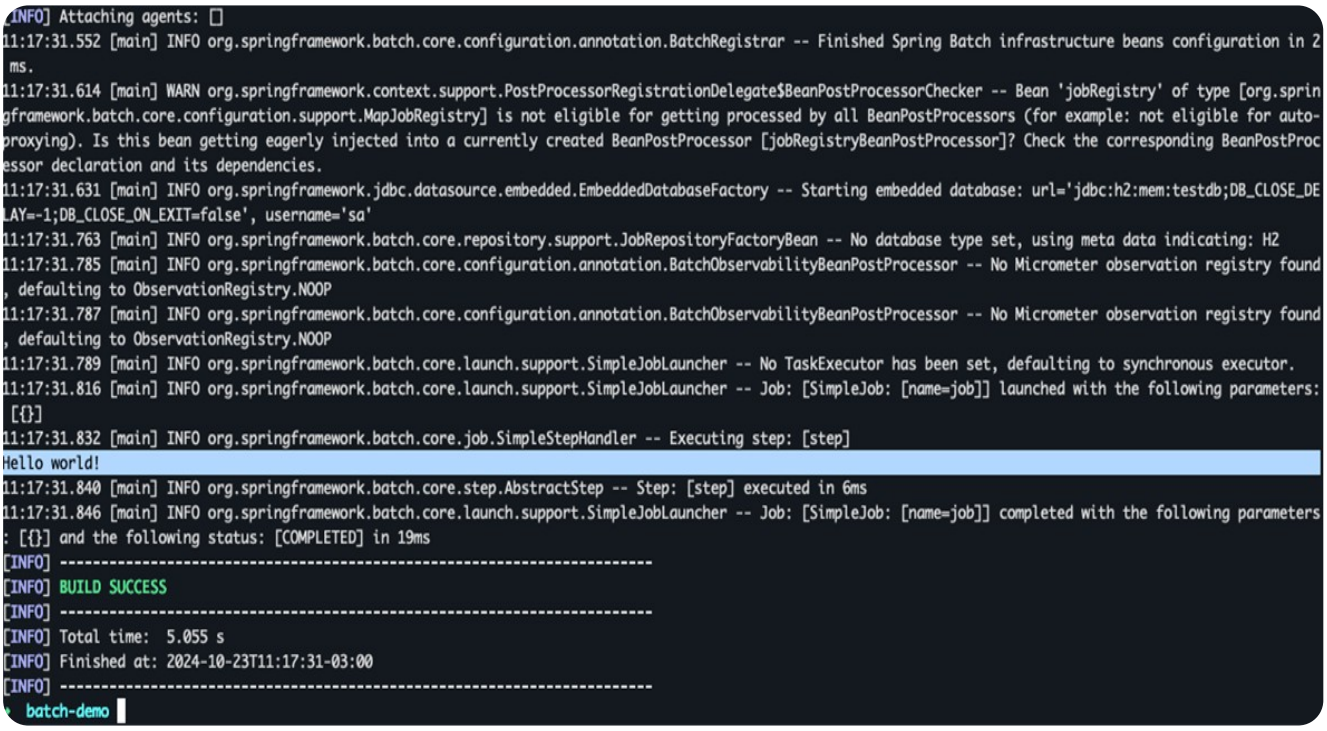
Ya sea que ejecutes tu primer proyecto o el proyecto batch_hello de referencia brindado anteriormente, obtendrás información de la ejecución. Para poder llegar a ejecutar tu proyecto deberás utilizar terminal Bash o PowerShell.

La Figura 6 muestra la salida de la ejecución de un trabajo batch en Spring Batch en la línea de comandos. Durante la ejecución, se observa que Spring Batch inicia la configuración del contexto, carga los beans necesarios y establece una base de datos embebida para persistir la información del trabajo.

El trabajo es lanzado con un nombre específico ([name=job1]) y ejecuta un paso (Step) que se completa en 6 milisegundos. Al finalizar, el trabajo muestra un mensaje de "BUILD SUCCESS" indicando que la ejecución fue exitosa, con un tiempo total de 5.055 segundos y la fecha de finalización correspondiente.

Figura 6

Output de hello world



Nota. La imagen muestra la ejecución exitosa de un trabajo Spring Batch. *Microsoft (2006). Power Shell* (7.4.5). *[Software].* https://learn.microsoft.com/en-us/powershell/

Links de Interés

En el siguiente enlace podrás ver ejemplos de implementaciones con diferentes casos de uso de procesos batch: https://github.com/spring-projects/spring-batch/tree/main/spring-batch-samples

Leyendo, transformando y persistiendo datos

En el contexto de Spring Batch, el procesamiento y transformación de datos es una de las fases más cruciales, especialmente cuando se trata de migrar o integrar datos de sistemas legacy. Esta fase asegura que los datos leídos desde una fuente se modifiquen adecuadamente antes de ser almacenados o enviados a su destino final.



Importante

Para poder observar un proyecto funcional, utiliza https://ava.duoc.cl/bbcswebdav/xid-5074924_1 en el cual revisarás los objetos relacionados en cada etapa.

Los pasos o fases en el procesamiento de datos son:

Paso 1: Lectura o procesamiento de datos

El primer paso en el procesamiento es obtener los datos desde la fuente original. Esto se realiza mediante un ItemReader, que puede leer datos de diversos formatos, como bases de datos, archivos planos (CSV, XML, JSON) o servicios web.

Figura 7

Ejemplo de ItemReader

```java
@Bean
public FlatFileItemReader<Person> itemReader() {
    // Configuración del lector para leer el archivo CSV
    FlatFileItemReader<Person> reader = new FlatFileItemReader<>();
    reader.setResource(new ClassPathResource("data.csv"));
    reader.setLinesToSkip(1); // Saltar la primera línea (cabecera)

    // Definir el mapeo de cada línea del archivo CSV a un objeto Person
    DelimitedLineTokenizer lineTokenizer = new DelimitedLineTokenizer();
    lineTokenizer.setNames("id", "name", "age");

    BeanWrapperFieldSetMapper<Person> fieldSetMapper = new BeanWrapperFieldSetMapper<>();
    fieldSetMapper.setTargetType(Person.class);

    DefaultLineMapper<Person> lineMapper = new DefaultLineMapper<>();
    lineMapper.setLineTokenizer(lineTokenizer);
    lineMapper.setFieldSetMapper(fieldSetMapper);

    reader.setLineMapper(lineMapper);
    return reader;
}
```

Nota. Obtención de los datos desde la fuente original mediante un ItemReader. Microsoft (2006). *Power Shell* (7.4.5). *[Software].* https://learn.microsoft.com/en-us/powershell/

Paso 2: Transformación de datos

Una vez que los datos han sido leídos, deben transformarse para cumplir con las necesidades del nuevo sistema o formato. Este trabajo lo realiza el ItemProcessor, que aplica lógica de negocio para modificar, validar o enriquecer los datos.

Tipos de transformación comunes:

- Conversión de formatos: transformar fechas o números a un formato estándar.
- Normalización: alinear los datos a un formato común, especialmente útil cuando se migran sistemas legacy.
- Enriquecimiento: agregar o completar datos faltantes desde otras fuentes.

Figura 8

Ejemplo de ItemProcessor

```
@Bean
public FlatFileItemReader<Person> itemReader() {
    // Configuración del lector para leer el archivo CSV
    FlatFileItemReader<Person> reader = new FlatFileItemReader<>();
    reader.setResource(new ClassPathResource("data.csv"));
    reader.setLinesToSkip(1); // Saltar la primera línea (cabecera)

    // Definir el mapeo de cada línea del archivo CSV a un objeto Person
    DelimitedLineTokenizer lineTokenizer = new DelimitedLineTokenizer();
    lineTokenizer.setNames("id", "name", "age");

    BeanWrapperFieldSetMapper<Person> fieldSetMapper = new BeanWrapperFieldSetMapper<>();
    fieldSetMapper.setTargetType(Person.class);

    DefaultLineMapper<Person> lineMapper = new DefaultLineMapper<>();
    lineMapper.setLineTokenizer(lineTokenizer);
    lineMapper.setFieldSetMapper(fieldSetMapper);

    reader.setLineMapper(lineMapper);
    return reader;
}
```

Nota. Transformación de datos realizado con ItemProcessor. Microsoft (2006). *Power Shell* (7.4.5). *[Software].* https://learn.microsoft.com/en-us/powershell/

Paso 3: Escritura de datos

El paso final es guardar los datos procesados en la nueva estructura o sistema. Esto se realiza a través del ItemWriter, que puede escribir en bases de datos, archivos o enviar datos a APIs externas.

Figura 9

Ejemplo de ItemWriter

```java
@Bean
public FlatFileItemReader<Person> itemReader() {
    // Configuración del lector para leer el archivo CSV
    FlatFileItemReader<Person> reader = new FlatFileItemReader<>();
    reader.setResource(new ClassPathResource("data.csv"));
    reader.setLinesToSkip(1); // Saltar la primera línea (cabecera)

    // Definir el mapeo de cada línea del archivo CSV a un objeto Person
    DelimitedLineTokenizer lineTokenizer = new DelimitedLineTokenizer();
    lineTokenizer.setNames("id", "name", "age");

    BeanWrapperFieldSetMapper<Person> fieldSetMapper = new BeanWrapperFieldSetMapper<>();
    fieldSetMapper.setTargetType(Person.class);

    DefaultLineMapper<Person> lineMapper = new DefaultLineMapper<>();
    lineMapper.setLineTokenizer(lineTokenizer);
    lineMapper.setFieldSetMapper(fieldSetMapper);

    reader.setLineMapper(lineMapper);
    return reader;
}
```

Nota. Guardar los datos procesados en la nueva estructura o sistema realizado con ItemWriter. Microsoft (2006). *Power Shell.* (7.4.5). *[Software].* https://learn.microsoft.com/en-us/powershell/

Estrategias de migración de sistemas legacy

Migrar un sistema legacy a una nueva infraestructura suele implicar mover datos desde una base de datos o sistema existente a uno nuevo, realizando transformaciones en el proceso.

Los sistemas legacy pueden tener datos inconsistentes, formatos antiguos y estructuras difíciles de manipular, por lo que una solución flexible y robusta es esencial. Por ello, para cada uno de los pasos vistos en “Leyendo, transformando y persistiendo datos” debes identificarlos claramente:

1. Lectura de datos: ¿Dónde están los datos del sistema legacy? Resolver esta pregunta es clave para lograr armar la primera etapa de tu batch (lectura). Como base debes considerar:

- Base de datos relaciones/no relacionales.
- Lectura de archivos planos.
- Lectura de APIs o servicios.

La estrategia de lectura debe tener en cuenta el volumen de datos y posibles inconsistencias o errores en los datos legacy. Una lectura eficiente puede ser clave para el éxito de la migración.

2. Transformación de datos: ¿Debo transformar algo en el proceso? (transformación). Esto puede incluir:

- Conversión de formato de datos.
- Normalización.
- Enriquecimiento de datos.
- Reclasificación de información.

3. Escritura de datos: ¿Dónde necesito la información para los nuevos sistemas? (Escritura). Algunas estrategias incluyen:

- Escritura en una nueva base de datos.
- Escritura en un sistema NoSQL.
- Escritura en APIs o servicios externos.



Importante

Es fundamental que la arquitectura contemple el manejo de grandes volúmenes de datos y posibles cuellos de botella al escribir los datos, aplicando técnicas como particionamiento o paralelización para mejorar el rendimiento.

Estrategias específicas para migrar datos legacy

Para diseñar una arquitectura eficiente en la migración de datos legacy, es importante aplicar ciertas estrategias específicas:

- Particionamiento y paralelismo: cuando los volúmenes de datos son muy grandes, es recomendable particionar el proceso, dividiendo la carga entre varios nodos o hilos para aumentar la velocidad del procesamiento.
- Manejo de datos inconsistentes: los sistemas legacy a menudo contienen datos con problemas. Spring Batch permite configurar políticas de omisión (SkipPolicy) o reintento (RetryPolicy) para manejar excepciones en el procesamiento.
- Normalización de datos: los ItemProcessors pueden encargarse de transformar datos no estructurados o inconsistentes en un formato estructurado y estándar.

Links de interés

Para levantar tu primer proyecto con Spring Batch considera utilizar https://ava.duoc.cl/bbcswebdav/xid-5074926_1

En el siguiente link podrás revisar ejemplos de casos de uso Spring Batch: https://github.com/spring-projects/spring-batch/tree/main/spring-batch-samples

Cierre de la semana



Recordemos que Spring Batch es un framework que permite procesar grandes volúmenes de datos de manera eficiente a través de trabajos por lotes (batch processing). Está diseñado para ejecutar tareas repetitivas en una secuencia de pasos definidos, garantizando la integridad de los datos y la optimización de recursos. Sus características principales incluyen la modularidad, reinicio de jobs, manejo de transacciones, escalabilidad y manejo robusto de errores. Surgió como una solución a los desafíos presentados en la migración de sistemas legacy, ofreciendo un enfoque más estructurado y eficiente para procesar y transformar datos.

La guía de esta semana compara Spring Batch con los enfoques tradicionales de procesamiento de datos, como los scripts personalizados, destacando las ventajas de una arquitectura modular y transaccional. El procesamiento manual presenta problemas de escalabilidad, mantenimiento y tolerancia a fallos, mientras que Spring Batch permite dividir los trabajos en pasos, reiniciar procesos fallidos y escalar de manera eficiente.

Spring Batch ha evolucionado para integrarse con tecnologías modernas, como Spring Boot, facilitando la migración de sistemas antiguos y permitiendo una mejor administración de los trabajos por lotes. Herramientas como estas son esenciales para gestionar procesos complejos en entornos empresariales modernos. La guía también incluye actividades formativas y recursos adicionales que te permitirán aplicar Spring Batch en proyectos reales, mejorando la eficiencia y confiabilidad en la migración y transformación de datos legacy.

Referencias

- Spring. (s.f.) Spring Batch Introduction. https://docs.spring.io/spring-batch/reference/spring-batch-intro.html
- Spring. (s.f.). Spring framework documentation. https://docs.spring.io/spring-framework/reference/
- Terasoluna (s.f.). Spring Batch Architecture. https://terasoluna-batch.github.io/guideline/5.0.0.RELEASE/en/Ch02_SpringBatchArchitecture.html

Lecturas de la semana

- Spring (s.f.). Chapter 3. The Domain Language of Batch. *Spring.* https://docs.spring.io/spring-batch/docs/2.2.x/reference/html/domain.html
- Spring (s.f.). Spring Batch architecture. *Spring.* https://docs.spring.io/spring-batch/reference/spring-batch-architecture.html

Apuntes



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.